

ACTS: Automatic Record-Level Incremental Computation for Unmodified Scientific CLIs

Joshua Terranova
University of Southern California
Los Angeles, California, USA

Abstract

Whole-command caches rerun an unmodified binary when any input byte changes. Hand-built per-record caches exist for one tool at a time. Nothing occupies automatic record-level reuse *inside one invocation* of an unmodified scientific CLI, persisted across runs. ACTS infers records and field roles of a VCF or FASTA→table binary with no per-tool code, memos those records, and ships a reassembly only when probes (and, in deployment, a Dune-style hit audit) pass. On SnpEff the later-run speedup is real and small: median $1.1668\times$ (95% CI [1.1595, 1.1705]) on an exclusive Linux node, with body MATCH on all 52,638 records. Startup caps the tool at about $1.3\times$ even at 100% hits. The deployed probe suite’s unsafe-ship rate is a *measurement*, not a new testing theory: 68/288 overall and 4/224 excluding the F6 environment/file faults the method does not claim to catch. HMMER collection savings, running baselines, and a head-to-head with command-grain incrementalizers remain **TODO**.

CCS Concepts

• **Software and its engineering** → **Software performance**; • **Applied computing** → *Bioinformatics*.

Keywords

incremental computation, memoization, black-box tools, scientific CLIs, VCF, HMMER

ACM Reference Format:

Joshua Terranova. 2026. ACTS: Automatic Record-Level Incremental Computation for Unmodified Scientific CLIs. In *Proceedings of Laboratory draft; not submitted (Draft)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Scientific command-line tools often map each record of a file—a VCF variant, a FASTA sequence—to an output row, independently of its neighbors. A later sample shares many of those records with an earlier one. Whole-command incrementalizers (Rattle [21], Riker [6], ProcessCache [20], and INCR [25]) treat the invocation as one unit: one new record in a large file is a changed input, so the process reruns. Hand-built per-record caches (eggNOG-mapper `-m cache` [3, 11], Oculus [24], SeAIM [22]) do the useful thing for one grammar and do not infer field roles of an unmodified third-party binary.

The gap we claim is narrow. **Nothing occupies automatic record-level reuse inside one invocation of an unmodified scientific CLI, persisted across runs.** Whole-command incrementalization of unmodified binaries is occupied. Hand-built per-record

caches are occupied. Safety *theory* is occupied (Chen/Segura meta-morphic relations [4, 18]; Dune’s cache-check audit [9]; vCache’s δ [17]). Cost-model gating is engineering, not a prior theorem.

ACTS is a wrapper, not a rewrite. On the first run of a given argv it probes a sample of records: determinism, shuffle, subset-invariance, field-role perturbation, late keys. The probes produce a cache key and a reassembly contract. Later runs pay the tool only for misses, reassemble, and—in the deployed mode—accept on probes plus a Bernoulli hit audit rather than a second full stock run. Byte-identical cmp is the VCF-body gate; HMMER tblout is token identity after whitespace normalization, because the format pads columns.

Three claims, judged against the related-work read (§4; [25] is the sharpest 2026 neighbor):

- (1) **Automatic record and field-role inference, no per-tool code — survives**, with a named neighbor. KumQuat infers *combiners* [19]; ProcessCache / Rattle / Riker / INCR infer file dependencies of a process or command; PaSh/POSH *declare* statelessness [13, 23]; eggNOG/Oculus/SeAIM hard-code one grammar. INCR’s optional chunking still needs a crowdsourced “stateless” / “argument-independent” annotation and applies to Unix stream utilities, not inferred CHROM, POS, REF, ALT versus INFO/ANN. What we own is *per-format* probes that produce a cache key and a reassembly contract for an unmodified VCF or FASTA→table binary.
- (2) **Safety probes with a measured miss rate in deployment mode — the measurement survives; the theory does not.** Our shuffle / subset / perturbation probes *are* meta-morphic relations [4, 18]. Dune occupies random re-exec plus diff [9]. vCache occupies a user-set δ for semantic LLM caches [17] (different object, stronger claim). What we own is the *measured* unsafe-ship rate of *this* probe suite under `-verify audit` (probes plus 2% hit audit, no full MATCH) on faults F1–F8 / controls C1–C5. Full-MATCH is an experimental upper bound, not the deployed number. We do not claim a δ , and we do not say “first statistical cache.”
- (3) **Cost-model gating that predicts speedup before caching — survives as engineering, not as a literature gap worth a paper by itself.** We do not lead with this. It is the screen, not the method: $t = a + b \cdot n$, refuse if the predicted ratio cannot pay for wrapper overhead w .

STAR [8] is a kill test for the *aligner instance*, not for the method: BAM is not a per-read function of the FASTQ, and Suite B cross-sample overlap is rare (human airway `recall_in_new` 0.067–0.108, nf-core 0.087, fly 0.017–0.020). Within-file uniqueness tops out at $1.15\times$ on i03. We do not claim we invented align-uniques-once; Oculus already occupies that instance [24].

Ensembl VEP’s official `-cache` is a *reference-data* store (transcripts, known variants), not a memo of prior user VCFs [15]. Adding sample 1,001 still annotates every variant in that VCF. We do not treat it as a competitor.

Contributions. (1) Per-format inference of records and field roles for unmodified VCF and FASTA→table CLIs (§3.1). (2) A probe suite whose deployed unsafe-ship rate is measured, with theory explicitly disclaimed (§3.2–§3.3). (3) A two-parameter cost screen used to pick tools, not sold as novelty (§3.4). (4) An evaluation plan that separates committed measurements from predictions and from work that has not run (§5).

2 Background

2.1 Scientific CLIs as per-record maps

A typical annotator reads a file of records and writes a file of the same cardinality, or a table with zero or more rows per input record. SnpEff [5] annotates each VCF variant. HMMER [10] searches each protein against a profile database. If record *i*’s output depends only on record *i* (plus argv-named inputs: binary, flags, database files), then a later file that repeats keys can reuse stored outputs.

Independence is an empirical property, not a type. A tool that writes the file length into every row, or that copies a neighbor’s key, is not per-record. STAR’s BAM carries QNAME, alignment coordinates, and headers that are not a function of the read sequence alone; ACTS refuses that argv on identity grounds.

2.2 Formats we handle

VCF. A record is a non-# line. The default correspondence key is CHROM POS REF ALT (full ALT). Output order follows input order. MATCH is the record *body*, not full-file cmp: SnpEff rewrites ##SnpEffCmd on every run. On 200 HG00096 -c1 variants, two stock runs and a shuffled run MATCH on the body; full-file identity fails on that header line only.

FASTA→table. A record is one sequence. The cache key is the amino-acid string. The query-name column is inferred, not hard-coded. Some tools emit rows in input order (hmmscan); others globally sort (hmmsearch). The contract records order versus multiset MATCH. HMMER -tblout pads columns and right-aligns numbers, so MATCH is whitespace-normalized (split()), not bytes.

2.3 The grain of reuse

Workflow managers (Nextflow -resume [7], Snakemake’s skip and -cache [16], Cromwell call caching [2]) hash a *task* and its declared inputs. They are the right answer when the scientist already split the work into processes. They do not look inside snpeff -i cohort.vcf. Interpreter-level memo (IncPy [12], Mandala [14]) cannot wrap a closed binary. Process-grain caches skip or rerun the whole exec-unit. ACTS sits between those grains: inside one invocation, at record resolution, without modifying the tool.

3 Method

ACTS wraps an unmodified argv. Example: python3 -m acts run -kind vcf -input X - <tool>. {input} in argv is replaced with a file the wrapper wrote (probe subset, miss batch, or one-record header harvest). Stdout is the format the kind declared. That

is format wiring, not a per-tool parser. snpeff_ann.py is a test oracle, not the runtime path.

A persistent JSONL cache is keyed by a *namespace* (kind, argv strings, content hash of the binary, content hash of every argv-named path) and a *record key*. Files the tool opens that do not appear in argv are out of scope unless probe-time tracing saw them (§3.2). Environment variables are a stated limitation (fault F6-env).

3.1 Record and field-role inference

Inference is per *format*, not per tool. The first run of a namespace draws probe_n records (code default 500; drawn without replacement, seed 20260927 in the deployed path) and classifies every output column / INFO subfield:

PASS-THROUGH equal to the (possibly perturbed) input value; taken from the new query record at reassembly.

PRODUCED unchanged across perturbation and not present in the input; cached, keyed by the current cache key.

DEPENDS-ON-NON-KEY changed, but not equal to the input; widen the cache key once to include the perturbed input field(s) and re-probe. Still ambiguous → REFUSE_AMBIGUOUS.

VCF.. Keys start as CHROM, POS, REF, ALT. Non-key fields (ID, QUAL, FILTER, INFO subfields, FORMAT, samples) are perturbed. A rewrite (output ≠ input, stable across perturbation) is PRODUCED, never silently treated as pass-through. Late INFO keys that appear only on a later miss batch trigger a bounded re-probe. Generic VCF inference replaced a SnpEff-specific splicer (commits 8540926, 3f47a00).

FASTA→table. The query-name column is the column that follows the FASTA header name under perturbation. Sequence is the only cache-key field. Columns that still depend on the description after one widen attempt (there is no extra FASTA field to add) refuse. Empty outputs (no domain hits) are cached as EMPTY so a later genome with the same sequence and a new locus tag is a hit, not a miss. Local HMMER on 200 K-12 proteins SHIPs both hmmscan -cut_ga (order MATCH) and hmmsearch with Z and domZ fixed (multiset MATCH). A reuse test on 500 proteins with 202 shared sequences SHIPs both modes after whitespace-normalized MATCH; 192/202 scan hits and 190/202 search hits were EMPTY, so NON-EMPTY reuse was not the bulk of that fixture.

Default hmmsearch E-values scale with the number of targets and fail subset-invariance; the locked argv therefore fixes Z. That is a flag choice the screen is allowed to make, not a per-tool parser.

3.2 Probes

On the probe sample:

- (a) **Determinism.** Run twice; differing bodies refuse as nondeterministic.
- (b) **Shuffle.** Permute records; differing bodies refuse as neighbor leakage.
- (b2) **Subset-invariance.** A singleton (or small subset) must match the corresponding rows of the full probe; else refuse as a global (file-level) dependence.
- (c) **Field-role perturbation.** As in §3.1.
- (d) **Late keys.** Unseen INFO keys / extra table columns on a miss batch trigger a bounded re-classification.

These are metamorphic relations in Chen and Segura’s sense [4, 18]. We do not claim a new MR theory. A program can satisfy the MRs and still be wrong; that is why MATCH (experimental) and the hit audit (deployed) sit behind the probes.

STAR argv is a special-case identity refuse from a pre-registered check, not from these probes.

Probe-time tracing. On Linux, the first probe invocation may run under `strace -f -e trace=openat,open`. Regular files opened read-only (excluding `/proc`, `/sys`, `/dev`, the per-run input, and temp dirs) are fingerprinted into the namespace. This is Rattle’s traced-input idea transferred to the first infer run, not a new key theory [21]. macOS, or `strace` missing: no tracing; F6-file is uncovered. Tracing is probe-time only, not every later run.

3.3 Hit audit

Dune’s cache-check-probability re-executes randomly chosen *build rules* and diffs them [9]. ACTS `-verify audit` (the default) is that idea at record grain: Bernoulli $p = 0.02$ of cache hits, then pad to at least 20 hits (or all hits), one batch, seed 20260927. A disagreement is REFUSE_AUDIT. This is engineering transfer, not a theorem, and not vCache’s user-set δ on a semantic LLM cache [17].

`-verify full` still runs a stock tool on the whole input and requires MATCH. That number is an experimental upper bound (§5). Deployment does not pay for it.

3.4 Cost model

Stock wall time on random subsets is fit as $t = a + b \cdot n$ (OLS on per-size mean wall). Wrapper overhead w is measured (full N , tool replaced by a no-op that writes empty-but-valid output). For a later-run miss fraction m ,

$$\begin{aligned} \text{ceiling}(m) &= \frac{a + b \cdot N}{a + b \cdot m \cdot N + w}, \\ \text{saved}(m) &= (a + b \cdot N) - (a + b \cdot m \cdot N + w). \end{aligned}$$

A tool advances the headline screen only if $a > 0$, $b > 0$, $\text{ceiling}(m) \geq 3.0$, and $\text{saved}(m) \geq 60$ s, with m the *measured* miss fraction on a locked later input—not a hoped-for 0.2.

This is the screen, not the paper’s method. Workflow papers that predict *memory or runtime for scheduling* (Ponder / KS+ [1]) are a different question. ProcessCache reports overheads after the fact. We do not claim a literature gap here.

On SnpEff, a is large relative to $b \cdot N$. After the exclusive Linux alternating run, $a_{\text{now}} = 15.98$ s predicts 1.203× versus measured median 1.167×; the same fit caps the tool at about 1.3× even at 100% hits. Heavier SnpEff (stats on, `-ud 20000`) fails the advance rule: $\text{ceiling}(0.2) = 1.30\times$. `ruff check -no-cache` passes the locked formula at 4.52× but is rejected: `ruff` already caches per file, `-no-cache` is a strawman, full-run wall is 0.705 s, and the first formula omitted w .

4 Related Work

Automatic incremental reuse is an old idea. The question is the grain, and whether the tool must be changed. This section follows the deep read in pipeline/docs/RELATED_WORK.md (§3); we do not add a fourth claim.

At process or command grain, several systems already wrap **unmodified** binaries. Rattle traces each build command and skips it when argv, environment, and hashed file inputs match [21]. Riker extends that model to the POSIX filesystem—directories, pipes, links—so a spec as short as `gcc *.c` still rebuilds correctly [6]. ProcessCache generalizes the same idea off the build system and onto arbitrary Linux process trees [20]. It memos **exec-units** (from `execve` to the end of the descendant tree), computes preconditions from a syscall log, and skips when those preconditions hold. On unchanged inputs it reports up to 65×; when a few input *files* change, speedup collapses, and a changed FASTA is one file. HMMER under ProcessCache is one process.

INCR [25] is the 2026 form of this line, and the closest neighbor: bolt-on incrementalization of unmodified **shell programs**, OverlayFS isolation plus `strace`, reuse of command effects across edits. With optional PaSh/POSH annotations [13, 23] it can chunk stdin of a *declared* stateless utility or split per argument. Default grain remains the command. A lone `hmmsearch` on a proteome is one command; one new sequence reruns it. Chunk memo needs that crowdsourced “stateless” annotation; it does not infer VCF/FASTA field roles. INCR does **not** occupy claim 1. We use their tracing idea at **probe time** to fingerprint files the tool opens. We do not claim first automatic cache keys.

Workflow managers occupy the next-coarser grain. Nextflow `-resume` [7], Snakemake’s intra-workflow skip and `-cache` [16], and Cromwell call-caching [2] all hash a **task** and its declared inputs. They do not look inside `snpEff -i cohort.vcf`.

Finer grain exists if the language is under control. IncPy [12] memos Python functions inside a modified CPython. Mandala [14] does the same with `@op` annotations. Neither reaches a closed binary.

Per-record reuse for *one* scientific tool is also occupied, and we should say so. Oculus collapses duplicate reads before Bowtie/BWA and reprints SAM; it is nearly lossless, not byte-identical [24]. SeALM caches alignment queries [22]. eggNOG-mapper’s `-m` cache is the cleanest neighbor: `-md5` then a user-supplied annotations file, exact amino-acid identity, misses re-searched [3, 11]. That is a hand-built memo of prior user sequences—our job, for one tool. Ensembl VEP `-cache` is the trap citation: it is a **reference-data** store, not a memo of the user’s last VCF [15].

Black-box *structure* inference without caching is KumQuat [19]: combiners for data-parallel Unix commands, 113 of 121 unique commands, no cross-run store. PaSh and POSH annotate the same commands for parallelism [13, 23]. We infer a different contract (which fields are keys, which are produced, whether neighbors leak) so we can memoize.

Correctness arguments for caches are also occupied, at other grains. Dune’s cache-check-probability re-executes random **build rules** and diffs them [9]. vCache [17] gives a user-set error bound δ for *semantic* LLM caches under i.i.d. prompts and a sigmoid model—a stronger claim than we can make, on a different object. Our shuffle, subset, and perturbation probes are metamorphic relations [4, 18]. The paper’s correctness contribution is not a new MR theory and not a δ . It is a **measured** unsafe-ship rate of this probe suite in the mode we actually ship (`-verify audit`: no full stock run; 2% hit audit, at least 20 hits). The earlier full-MATCH number is an experimental upper bound.

Table 1: Probe-eval unsafe-ship rates. Full-MATCH is an experimental upper bound; `-verify audit` is the deployed number. F6-env is outside the `argv-named-input` guarantee. F6-file requires Linux tracing, which this Mac run did not have.

Metric	<code>-verify full</code>	<code>-verify audit</code>
Unsafe-ship overall	33/256	68/288
Excl. F6 / F6-env+F6-file	1/224	4/224
F6-env (limitation)	32/32	32/32
F6-file (macOS, no strace)	—	32/32
False-refuse (C1–C5 × 4)	0/20	0/20
Audit-only catch	—	12

What is left, and what we claim, is narrow: infer records and field roles of an unmodified VCF or FASTA→table binary with no per-tool code; memoize those records across runs; refuse when the probes fail; and, in deployment, accept on probes plus a Dune-style audit rather than a second full run. ProcessCache, Rattle, Riker, and INCR will rerun the process when any input byte changes. eggNOG will not annotate a SnpEff VCF. KumQuat will parallelize sort and leave tomorrow’s proteome uncached.

5 Evaluation Plan

We separate (i) committed measurements, (ii) model predictions, and (iii) experiments that have not produced a result file on this branch. No number below is taken from uncommitted dirty-main artifacts.

5.1 Already measured

SnpEff later-run speedup (reference). SnpEff 5.4c (GRCh38.86; `-noStats -noLog`) on HG00099 -c1 ($N = 52,638$). Exclusive Linux node b22-02 (AMD EPYC 7542), alternating stock vs cached, ten pairs. Median wall ratio 1.1668× (95% bootstrap CI [1.1595, 1.1705]; seed 20260924, 10,000 resamples). All ten pairs have equal bodies. MATCH on all 52,638 reassembled records versus stock. A same-day Mac run reported 1.29× and is *withdrawn as a headline*: the idle precondition failed (load 5 → 27). It remains a recorded result, not the claim.

1000 Genomes CEU chr22 -c1 overlap: pairwise recall_in_new 0.620–0.637; after two samples the third is already 0.787 seen. Overlap is not a speedup.

Probe miss rate. Synthetic faults F1–F8 and controls C1–C5, both VCF and FASTA, $N_{\text{rec}} = 2500$, probe_n ∈ {50, 200, 500, 2000}, seed 20260927. Protocol locked in PROBE_EVAL_PROTOCOL.md (b4c878f; audit addendum 4679264).

In-scope unsafe-ship (56 cells per probe_n: 7 classes × 4 frequencies × 2 formats, excluding F6-env and F6-file): 3/56 at 50, 1/56 at 200, 0/56 at 500, 0/56 at 2000. The post-hoc recommendation is probe_n = 500 (about 504 first-infer tool calls). That recommendation was not locked in b4c878f or 4679264. The code default is already 500; the savings runner used 8 (§6).

HMMER stock screen (no cache, no speedup claim). Exclusive-node stock $t = a + b \cdot n$ on BW25113 ($N = 4,192$), HMMER 3.4, Pfam-A 38.2, -cpu 32. Both modes ADVANCE the locked rule. paper_uses =

Table 2: Predicted later-genome speedup from $t = a + b \cdot n$ and median miss fraction $m(k)$ over 20 orderings. Not a timed result.

Collection	Mode	$k=10$	$k=30$	$k=39$
A (diverse <i>E. coli</i>)	hmmsearch	2.96×	4.10×	4.87×
A	hmmsearch (primary)	2.12×	2.52×	2.73×
B (O157:H7)	hmmsearch	36.7×	59.8×	50.7×
B	hmmsearch (primary)	4.47×	4.65×	4.60×

hmmsearch (first mode that passes; if both, hmmsearch). K-12 miss fraction $m = 0.0019$ is a best case, not a diverse-collection claim.

5.2 Predictions (not measurements)

The Part 2 model applies the screen fit to recurrence curves with N locked at the screen proteome 4,192, not per-genome N_i . Numbers in this subsection are **predictions**.

Cumulative predicted wall on the locked savings workloads (30 genomes of A, 40 of B; $N = 4,192$ stand-in): A/hmmsearch cached 31,288 s vs stock 91,865 s (predicted cum. 2.94×); A/hmmsearch predicted cum. 2.11×. A 3× claim on hmmsearch from this experiment is not licensed by the model on collection A.

5.3 Not yet measured

- **HMMER accumulated savings.** Sequential stock-vs-cached wall and CPU over collections A and B, both modes, MATCH plus Dune-style audit (pre-registered). **TODO: jobs pending; no savings result JSON on this branch.**
- **Running baselines.** The tool’s own cache (eggNOG -m cache on the HMMER instance; SnpEff has none at record grain); Riker; ProcessCache; INCR on the same argv. **TODO: comparison not run.**
- **Linux F6-file and snpEff.config namespace.** Protocol requires Linux strace. The Mac audit run left this INCOMPLETE. **TODO**
- **VEP.** Protocol locked; binary and cache not on disk. **TODO**

6 Threats to Validity

Savings jobs used probe_n=8. The HMMER savings runner hard-codes PROBE_N = 8, while probe-eval (post-hoc) recommends 500 to reach 0/56 in-scope unsafe-ship. A collection result, when it lands, is evidence under the smaller probe, not under the recommended size.

Whitespace-normalized MATCH for tables. HMMER tblout is not byte-identical across equivalent runs (column padding, right-aligned numbers). The paper claims token identity (split()), order MATCH for hmmsearch -cut_ga and multiset MATCH for hmmsearch with fixed Z. A reviewer who expects cmp will call this a weaker oracle. It is the declared oracle, not a silent switch after a mismatch.

F6-env. A tool that reads ACTS_PROBE_EVAL_ENV (or any environment variable not in argv) is undetectable by construction. All 32 F6-env cells unsafe-ship. This is a limitation of argv-named-input namespaces, not a probe bug.

Tracing only on Linux. Probe-time strace is how F6-file (a hidden config next to the input) enters the namespace. The committed audit run is macOS: trace_status=unavailable, F6-file 32/32 unsafe-ship. Linux F6-file plus a snpEff.config check is INCOMPLETE.

Mac versus exclusive-node timing. SnpEff Mac 1.29× broke its idle precondition and is not the claim. The reference is the exclusive Linux alternating run (1.1668×). HMMER screen a/b were fit on a different exclusive node (b22-16) than the SnpEff run (b22-02); both are EPYC 7542. Savings jobs, when they run, interleave stock and cached on one exclusive node to avoid that confound. Do not mix Mac wall times with Linux speedups.

hmmScan is a declared post-hoc deviation. HEADLINE_SCREEN.md locked paper_uses = hmmsearch if both modes ADVANCE. Both did. The Part 2 model then showed hmmsearch is startup-capped on collection A, while hmmScan scales with recurrence. **Both modes are run.** hmmsearch stays the pre-registered primary. hmmScan is a post-hoc selection, justified by that model and by the official EBI pfam_scan.pl (retrieved 2026-09-27; PfamScan.pm sets the binary to hmmScan and default -cut_ga). A 3× sentence, if it appears, is licensed only for the mode the measured totals support, and hmmScan 3× on A would be post-hoc.

Other threats (brief). Probe-eval tools are small Python faults, not SnpEff/HMMER with a injected bug. Directory fingerprints hash metadata, not contents. save() is last-writer-wins. VEP is absent. Collection C is not in the savings run. Synthetic overlap on chr22 CEU is not a clinical cohort.

7 Conclusion

ACTS infers records and field roles of an unmodified VCF or FASTA-to-table CLI and memos those records across runs. The literature already occupies whole-command incrementalization, hand-built per-record caches, MR theory, Dune-style audits, and vCache-style δ bounds. The remaining object is automatic record-level reuse inside one invocation, persisted across runs, with a measured—not theorized—unsafe-ship rate.

SnpEff shows the method can MATCH and still be a bad headline (1.17×, startup-capped). Probe-eval shows the deployed audit mode misses 4/224 in-scope cells on this suite, and systematically misses env-hidden inputs. Whether FASTA→table reuse on recurring proteomes pays for the wrapper is the measurement that is still out: **TODO: HMMER savings, baselines, comparison.**

References

- [1] Jonathan Bader et al. 2023. Ponder / KS+: predicting memory and runtime of Nextflow tasks. *IEEE BigData 2023*, arXiv:2408.00047, arXiv:2408.12290. Scheduling predictors, not a record-cache gate.
- [2] Broad Institute. 2024. Cromwell call caching. <https://cromwell.readthedocs.io/en/stable/Configuring/>. Call caching; documentation.
- [3] Carlos P. Cantalapiedra, Ana Hernández-Plaza, Ivica Letunic, Peer Bork, and Jaime Huerta-Cepas. 2021. eggNOG-mapper v2: Functional Annotation, Orthology Assignments, and Domain Prediction at the Metagenomic Scale. *Molecular Biology and Evolution* 38, 12 (2021), 5825–5829. doi:10.1093/molbev/msab293
- [4] T. Y. Chen, S. C. Cheung, and S. M. Yiu. 1998. *Metamorphic Testing: A New Approach for Generating Next Test Cases*. Technical Report HKUST-CS98-01. Hong Kong University of Science and Technology. arXiv:2002.12543.
- [5] Pablo Cingolani, Adrian Platts, Le Lily Wang, Melissa Coon, Tung Nguyen, Luan Wang, Susan J. Land, Xiangyi Lu, and Douglas M. Ruden. 2012. A Program for Annotating and Predicting the Effects of Single Nucleotide Polymorphisms, SnpEff: SNPs in the Genome of *Drosophila melanogaster* Strain w1118; iso-2; iso-3. *Fly* 6, 2 (2012), 80–92. doi:10.4161/fly.19695
- [6] Charlie Curtisinger and Daniel W. Barowy. 2022. Riker: Always-Correct and Fast Incremental Builds from Simple Specifications. In *Proc. USENIX Annual Technical Conference (ATC)*. 885–898. <https://www.usenix.org/system/files/atc22-curtisinger.pdf>
- [7] Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. 2017. Nextflow Enables Reproducible Computational Workflows. *Nature Biotechnology* 35 (2017), 316–319. doi:10.1038/nbt.3820
- [8] Alexander Dobin, Carrie A. Davis, Felix Schlesinger, Jorg Drenkow, Chris Zaleski, Sonali Jha, Philippe Batut, Mark Chaisson, and Thomas R. Gingeras. 2013. STAR: Ultrafast Universal RNA-seq Aligner. *Bioinformatics* 29, 1 (2013), 15–21. doi:10.1093/bioinformatics/bts635
- [9] Dune Developers. 2024. Dune cache-check-probability. https://dune.readthedocs.io/en/stable/reference/config/cache_check_probability.html. Documentation.
- [10] Sean R. Eddy. 2011. Accelerated Profile HMM Searches. *PLoS Computational Biology* 7, 10 (2011), e1002195. doi:10.1371/journal.pcbi.1002195
- [11] eggNOG-mapper developers. 2021. eggNOG-mapper cache mode (~m cache). <https://github.com/eggnogdb/eggnog-mapper/wiki>. Product documentation; ~md5 then user-supplied annotations file.
- [12] Philip J. Guo and Dawson Engler. 2011. Using Automatic Persistent Memoization to Facilitate Data Analysis Scripting. In *Proc. ISSTA*. doi:10.1145/2001420.2001455
- [13] Konstantinos Kallas, Tammam Mustafa, Jan Bielak, Dimitris Karnikis, Thurston H. Y. Dang, Michael Greenberg, and Nikos Vasilakis. 2022. Practically Correct, Just-in-Time Shell Script Parallelization. In *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [14] Alexander Makelov. 2024. Mandala: Compositional Memoization for Simple & Powerful Scientific Data Management. In *Proc. Python in Science Conference (SciPy)*. <https://proceedings.scipy.org/articles/JHPV7385>
- [15] William McLaren, Laurent Gil, Sarah E. Hunt, Harpreet Singh Riat, Graham R. S. Ritchie, Anja Thormann, Paul Flicek, and Fiona Cunningham. 2016. The Ensembl Variant Effect Predictor. *Genome Biology* 17 (2016), 122. doi:10.1186/s13059-016-0974-4
- [16] Felix Mölder, Kim Philipp Jablonski, Brice Letcher, Michael B. Hall, Christopher H. Tomkins-Tinch, Vanessa Sochat, Jan Forster, Soohyun Lee, Sven O. Twardziok, Alexander Kanitz, Andreas Wilm, Manuel Holtgrewe, Sven Rahmann, Sven Nahnsen, and Johannes Köster. 2021. Sustainable Data Analysis with Snakemake. *F1000Research* 10 (2021), 33. doi:10.12688/f1000research.29032.2
- [17] Luis Gaspar Schroeder, Aditya Desai, Alejandro Cuadron, Kyle Chu, Shu Liu, Mark Zhao, Stephan Krusche, Alfons Kemper, Matei Zaharia, and Joseph E. Gonzalez. 2026. vCache: Verified Semantic Prompt Caching. In *Proc. International Conference on Learning Representations (ICLR)*. arXiv:2502.03771. doi:10.48550/arXiv.2502.03771
- [18] Sergio Segura, Gordon Fraser, Ana B. Sánchez, and Antonio Ruiz-Cortés. 2016. A Survey on Metamorphic Testing. *IEEE Transactions on Software Engineering* 42, 9 (2016), 805–824. doi:10.1109/TSE.2016.2532875
- [19] Jiasi Shen, Martin Rinard, and Nikos Vasilakis. 2022. Automatic Synthesis of Parallel Unix Commands and Pipelines with KumQuat. In *Proc. PPoPP*. doi:10.1145/3503221.3508400
- [20] Kelly Renee Shiptoski. 2023. *Reproducibility and Performance Optimizations for Unmodified Linux Programs*. Ph.D. Dissertation. University of Pennsylvania. <https://repository.upenn.edu/handle/20.500.14332/59482>
- [21] Sarah Spall, Neil Mitchell, and Sam Tobin-Hochstadt. 2020. Build Scripts with Perfect Dependencies. *Proc. ACM Program. Lang.* 4, OOPSLA (2020). doi:10.1145/3428237
- [22] Evan Stene and Farnoush Banaei-Kashani. 2019. SeAlM: A Query Cache Optimization Technique for Next Generation Sequence Alignment. In *Proc. IEEE International Conference on Data Mining Workshops (ICDMW)*. doi:10.1109/ICDMW.2019.00139
- [23] Nikos Vasilakis, Konstantinos Kallas, Konstantinos Mamouras, Achilles Benetopoulos, and Lazar Cvetković. 2021. PaSh: Light-touch Data-Parallel Shell Processing. In *Proc. EuroSys*.
- [24] Brendan A. Veeneman, Matthew K. Iyer, and Arul M. Chinaiyan. 2012. Oculus: Faster Sequence Alignment by Streaming Read Compression. *BMC Bioinformatics* 13 (2012), 297. doi:10.1186/1471-2105-13-297
- [25] Yizheng Xie, Evangelos Lamprou, Jerry Xia, and Nikos Vasilakis. 2026. Faster Re-execution via Bolt-on Incrementalization. In *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. <https://www.usenix.org/system/files/osdi26-xie-yizheng.pdf>